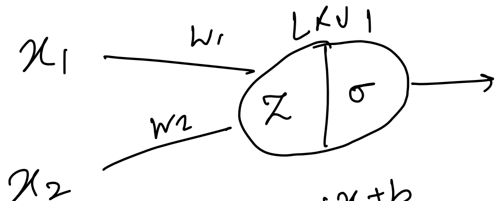
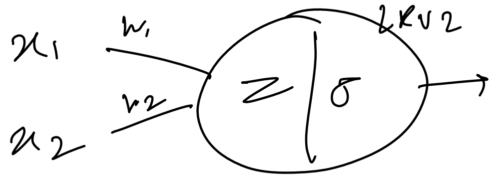
- Implementation of Neural network.
- Notations
- Softmax classifier
- Categorical Crossentropy loss
- Forward propogation
- Backward prob.

OVR One vs Rest

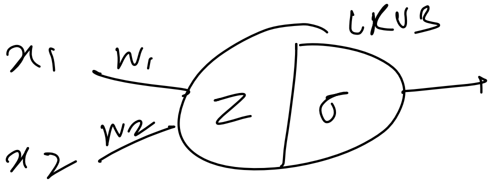


$P > 0.5 \rightarrow A$ else rest

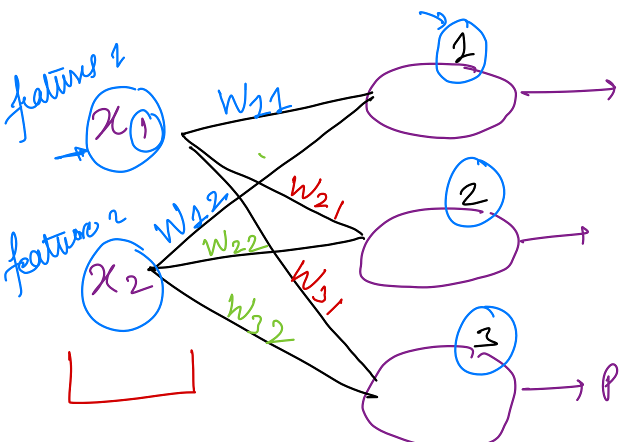
$z = wx + b$



$p > 0.5 \rightarrow B$ else rest



$p > 0.5 \rightarrow C$ else rest.



$p > 0.5 \rightarrow A$

$p >$

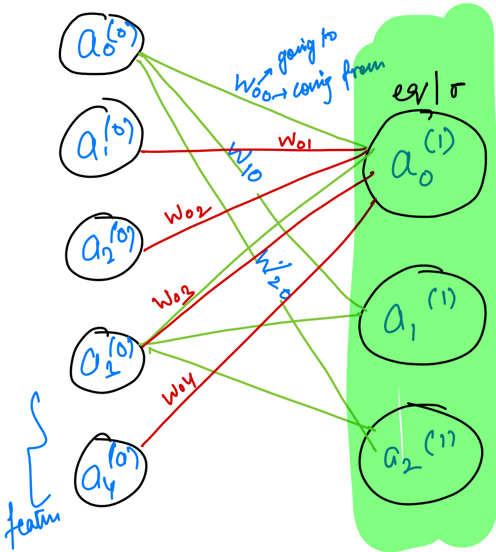
$$3 \times 2 \quad \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ \vdots & \\ \vdots & \end{bmatrix}$$

no. of features $= d = 2$

no. of inputs $= m = 300$

output $\Rightarrow 3$

n



i/p layer   o/p layer

Activation function

$$\underline{a_0^{(1)}} = \sigma\left(W_{00}a_0^{(0)} + W_{01}a_1^{(0)} + W_{02}a_2^{(0)} + W_{03}a_3^{(0)} + W_{04}a_4^{(0)} + b_0\right)$$

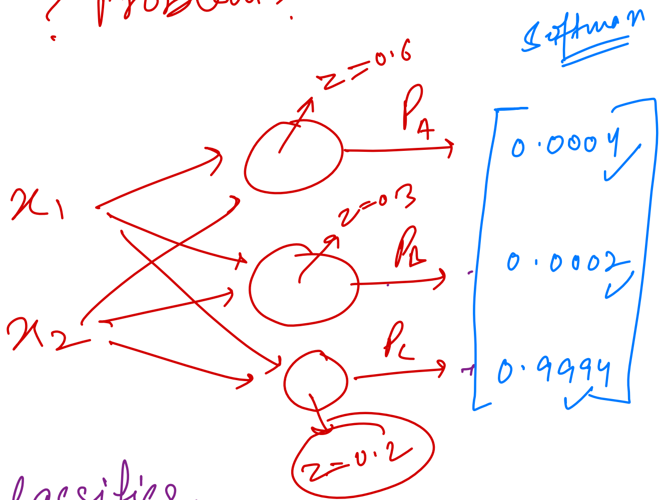
bias

$$\begin{bmatrix} W_{00} & W_{01} & W_{02} & W_{03} & W_{04} \\ W_{10} & W_{11} & W_{12} & W_{13} & W_{14} \\ \vdots & & & & \\ W_{n0} & W_{n1} & W_{n2} & W_{n3} & \dots W_{nd} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ a_2^{(0)} \\ a_3^{(0)} \\ \vdots \\ a_d^{(0)} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

$$a^{(1)} = \sigma\left(\underbrace{W a^{(0)} + b}_{z}\right)$$

$$a^{(1)} = \sigma(z)$$

Sigmoid ? Problem ?



Sigmoid $= \frac{1}{1+e^{-z}} = (—)$

$[0-1]$

$T(P) \neq 1$

Resist using sigmoid

$z = [-\infty, \infty]$

Softmax Classifier

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$p_i = \frac{e^{z_i}}{\sum_{i=0}^{n} e^{z_i}} \Rightarrow \frac{e^{0.6}}{e^{0.6 \rightarrow 2}+e^{0.3}+e^{0.2}}$$

$$\cancel{p_i = \frac{z_i}{\sum_{i=0} z_i}} \rightarrow \cancel{\frac{0.6}{0.6+0.3+0.2}}$$

Softmax function

⇒ Avoid −ve values as prob. → x → we give −ve values

→ $\frac{d e^x}{dx}$ ⇒ nicely differentiable

→ interpreted as log-likelihood

Softmax

How to train a neural network?

$d = 2$

m datapoints

f1	f2
0.03	0.04

$\times \begin{bmatrix} \quad \end{bmatrix} + $ [n×1] or (1×n) $\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ \vdots \end{bmatrix} =$

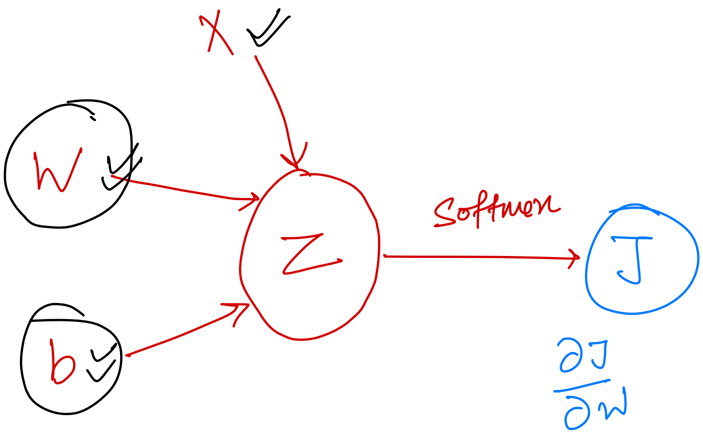
broadcasting? → m×n

n=3 (output)

A	B	C
0	0	1

$$\underbrace{\underset{\text{rows}\quad\text{column}}{m \times d} \times d \times n}_{} + \underbrace{m \times n}_{} = m \times n$$

$$[m \times n] + [m \times n]$$



$z = wx + b$

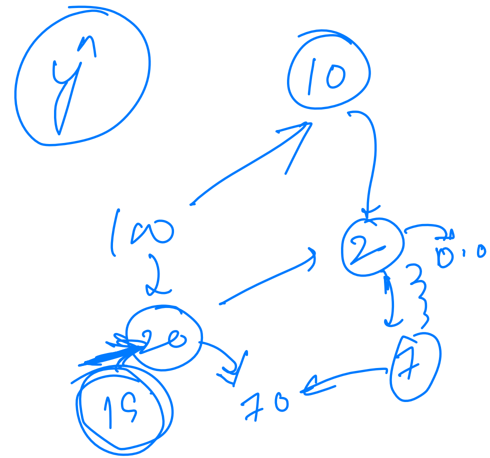
$\underset{\text{output}}{a} = \text{softmax}(z)$

$\hat{y} = \text{softmax}(z)$

$J = \hat{y} - y$ (reduce this diff / loss) w & b can be changed

$$W^{old} = W^{new}$$

$$W = W - \alpha \frac{\partial J}{\partial W}$$



$$\frac{\partial J}{\partial W}$$ ✓

loss function for classification

Categorical cross entropy

(Binary)

2 Classes

$$\left[-\frac{1}{n} \sum_{i=1}^{n} y_i \log(p_i) + (1-y_i)\log(1-p_i) \right]$$

2 Class

$y_i \to$ actual labels

$p_i \to$ predictions (probabilities)

Multi class

$$-\frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{n} y_{ij} \times \log(p_{ij})$$

i → points

j → no. of classes

Actual labels

	~~A~~ 0	~~B~~ 1	~~C~~ 2
1			
2	1	0	0
3			
4	1	0	0
.			
.			
300			

m=300

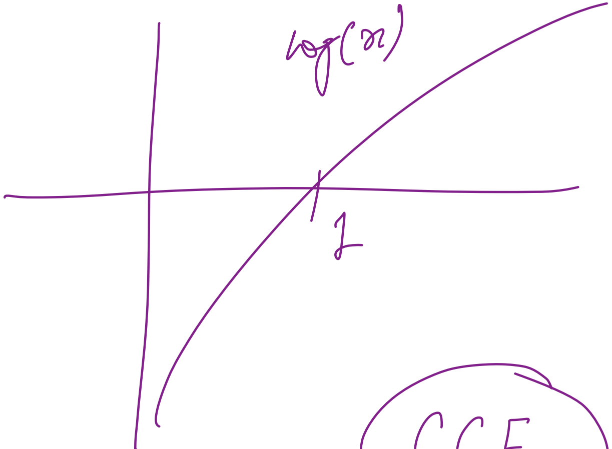
	0	1	2
0.999	1	0.001	0.001
4	0	1	0

when prediction is right

$$\text{loss} = -\frac{1}{1}\,(1)\log(1) + (0)\log(0.001) = 0 \sim \text{loss}$$

$$+ \quad 0 \quad + 0$$

$$\text{loss} = -1\,(1)\log(0) + \cancel{0\log(1)} \rightarrow 0$$



$$\Rightarrow -1 \times -\infty$$

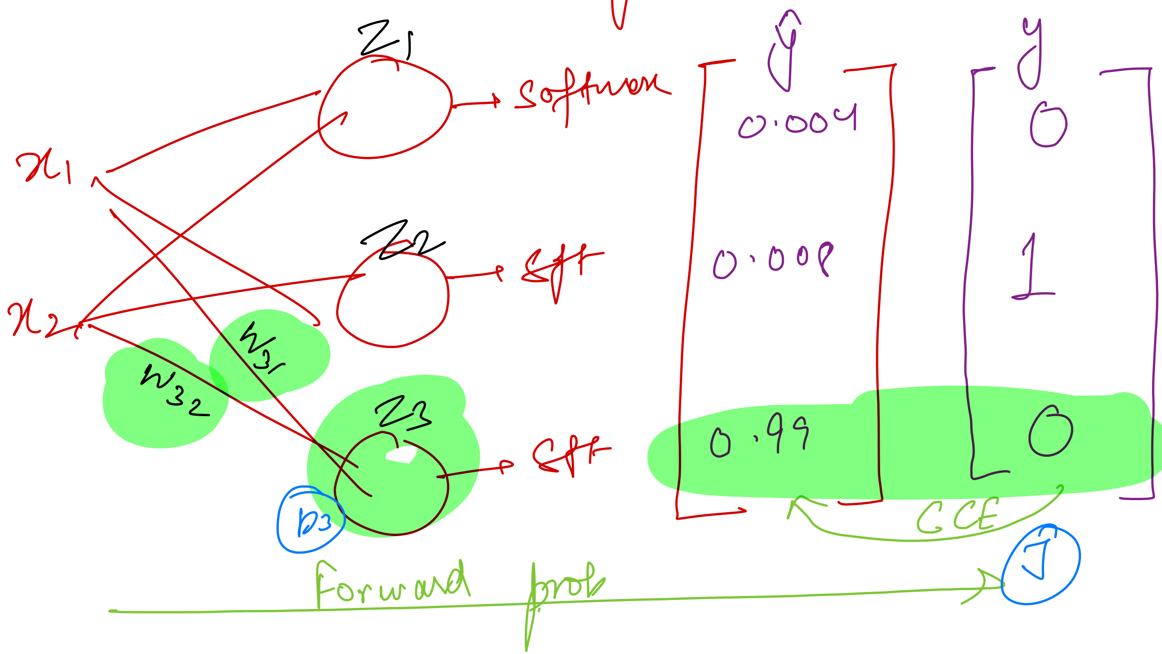
$$\Rightarrow \infty \Rightarrow \underline{\text{loss}}\ (\text{Max loss})$$

$$\text{CCE} = (\overset{\text{actual}}{y}, \hat{y})$$

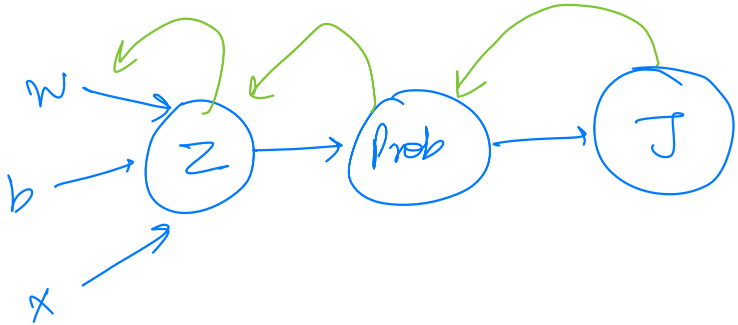
$$(y,\ \underbrace{\text{activation fn}(wx+b)}_{\text{forward propagation}})$$

loss (minimization)

gradient descent



$$W = W - \alpha \frac{\partial J}{\partial W}$$



$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial Prob} \times \frac{\partial Prob}{\partial Z} \times \frac{\partial Z}{\partial w}$$

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial Prob} \times \frac{\partial Prob}{\partial Z} \times \frac{\partial Z}{\partial b}$$

Chain rule of differentiation which is the backbone of NN.